Lab Worksheet: Tracing a Response from w.Write() to the Wire

Name: Kelvin Gordon & Teryn Guzman

Date: 16/10/2026

Write each prediction before you run the experiment, then record what actually happened.

Experiment 1: One small response (/hello)

Prediction

1. **Number of SOCKET conn.Write lines:** 1
2. **Appears before or after handler returns:** after
3. **Will the SOCKET line say 26, or something else:** It will say something else.

What actually happened

- **SOCKET conn.Write lines:** 1
- **Appeared after handler returns:** yes
- **Bytes on the SOCKET line:** 143 (26 body + 117 header)
- **Response header: Content-Length:** 26

Where were the bytes?

After this log line	Bytes in A	Bytes in B	Sent to kernel	Client has seen
handler starts	0	0	0	nothing
w.Write #1: 26 bytes	26	0	0	nothing
handler returns	26	0	0	nothing
SOCKET conn.Write: 143 bytes	0	0	143	The whole response

Explain in your own words

1. **Why does the response say Content-Length: 26 and not Transfer-Encoding: chunked?**

   When the chunkWriter built the header, the handler had already returned and all 26 bytes were still in Buffer A, which holds 2,048. Go therefore knew the exact body length and could write Content-Length: 26. Chunked is only needed when the length is unknown, which happens if A fills or is flushed before the handler finishes.

2. **Which log line is header moment 1, which is moment 3, and why is moment 2 not in the log?**

Moment 1 is the line first Write: net/http calls WriteHeader(200) for you; status fixed here. Moment 3 is the SOCKET conn.Write: 143 bytes line, because that is when the header reaches the kernel and the client can see it. Moment 2 (building the header bytes) happens inside Go, in the chunkWriter between A and B. Neither instrument sits there, so it leaves no log line. The only evidence is that the SOCKET line is 117 bytes bigger than the body.

3. **Which box held your 26 bytes when the handler returned, and what does that say about the phrase 'I already wrote the response'?**

   Buffer A. “I already wrote the response” only means the bytes were copied into a buffer. The client had received nothing at that point and only got data when the SOCKET line appeared.

Experiment 2: Two buffers filling in slow motion (/drip)

Prediction

1. **After Write #2 (2,000 bytes), has anything reached the socket? Why or why not?**
   No, because it hasn’t filled the total 2,048 bytes.

2. **Which Write makes Buffer A overflow first, and where do its bytes go?**
   Write #3 makes it overflow and the remaining amount goes to Buffer B.

3. **Which Write makes Buffer B overflow first? What will the first SOCKET line say (bytes, and what it starts with)?**
   Write #5 makes it overflow first, i am not sure what the line will start with

4. **After how many seconds does the client first receive anything?** After 1.2 seconds

5. **Content-Length or Transfer-Encoding: chunked?** chunked

What actually happened

- First byte after (s): 1.41 (curl). The log shows 1.2 s between Write #1 and the first SOCKET line.
- Finished after (s): 2.61
- SOCKET lines (bytes, in order): 4,096 (starts HTTP/1.1 200 OK, during the handler), then 4,062 (starts with dddd..., after handler returns)
- Header: Transfer-Encoding: chunked

Step 6: same handler with size=100

Prediction (first byte time, header type): first byte about 2.4 s, Content-Length.

Actual: first byte 2.62 s, finished 2.62 s, Content-Length: 800, one SOCKET line of 918 bytes (800 body + 118 header), after handler returns.

Explain in your own words

1. **Why did the 1,000-byte version use chunked and the 100-byte version Content-Length? Use the words Buffer A and the chunkWriter.**

   With 1,000-byte writes, Buffer A overflowed at Write #3 while the handler was still running, so the chunkWriter had to build the header before it knew the total length and chose chunked. With 100-byte writes, the whole 800 bytes stayed in Buffer A until the handler returned, so the chunkWriter knew the exact length and wrote Content-Length: 800.

2. **Same code, same pauses, same total time: why does the client see the first byte at different times?**

   In the 1,000-byte run, Write #5 pushed Buffer B past 4,096 bytes at 1.2 s, so the data reached the kernel during the handler and the client got its first byte then. In the 100-byte run, neither buffer ever filled, so nothing was sent until the handler returned at 2.4 s, and the client got everything at once.

3. **A page takes 3 seconds to build, written in small Write calls without Flush. Which box holds the data, and what two things could the developer do?**

   Buffer A holds the data, since small writes never fill it, so the client sees a blank screen until the handler returns. The developer could call Flush() after writing the part that's ready, such as the top of the page, so the client gets it early (at the cost of losing Content-Length and an extra system call). Or they could make the page build faster, for example by caching or doing the slow work in parallel, so there's less time to wait.

Experiment 3: Many small writes against one big write

Request	Write calls	Body bytes	Guess: SOCKET lines	Guess: Content-Length or chunked	Actual SOCKET lines (bytes)	Actual header
/pieces?n=100	100	1,000	1	Content-Length	1 line: 1,119	Content-Length: 1000
/pieces?n=300	300	3,000	1	Chunked	1 line: 3,144	Chunked
/pieces?n=1000	1,000	10,000	2	Chunked	3 lines: 4,096, then 4,096, then 1,973	Chunked
/big?kb=1	1	1,024	1	Content-Length	1 line: 1,143	Content-Length: 1024
/big?kb=64	1	65,536	1	Chunked	3 lines: 4,096, then 61,572, then 7	Chunked

Explain in your own words

1. **You called Write 1,000 times in pieces?n=1000 but the kernel received 3 calls. Where were the bytes in between? Name the boxes.**

   The bytes were held in Buffer A and Buffer B. Each 10-byte Write was copied into A, A emptied through the chunkWriter into B every 2,048 bytes, and B only emptied into the socket when it reached 4,096 bytes. So 1,000 small Writes became just two full-B writes during the handler and one final write for the remainder.

2. **In pieces?n=300 the header was built during the handler, but the SOCKET line came after handler returns. Explain using the sizes of A and B.**

   The 3,000-byte body is bigger than A (2,048), so A overflowed during the handler and the chunkWriter built the header then (moment 2), with chunked because the handler hadn't returned. That moved about 2,048 bytes plus the header into B. B holds 4,096 bytes and only had about 2,100 in it, so it never filled, and nothing reached the socket until the handler returned. Moment 2 and moment 3 are therefore different events.

3. **A friend says to join small strings into one big string before writing, to avoid many system calls. Is that necessary? When would it still be useful?**

   Not for avoiding system calls. The buffers already collect small Writes, so 1,000 Writes cost only 3 write() calls. It could still help in other ways, such as fewer function calls in a very hot loop, or having the whole body in memory to measure its length, check for errors, or set Content-Length yourself before writing anything. That connects to the safe pattern in Experiment 5.

Experiment 4: Flush (/hold)

Prediction (for flush=0 and flush=1, pause=1500)

1. **How long before the client receives its first byte?**
   For flush=0 the first byte is about 1.5 s; for flush=1 the first byte is about 0 s.
2. **Does the SOCKET line appear before or after the line sleeping 1.5s?**
   For flush=0 the SOCKET line appears after sleeping; for flush=1 the SOCKET line appears before sleeping.
3. **Content-Length or chunked?**
   For flush=0 it would be Content-Length; for flush=1 it would be chunked.

What actually happened

- **flush=0:** ttfb / total / header: ttfb 1.72 s, total 1.72 s, Content-Length: 100. The log shows sleeping 1.5s, then handler returns, then one SOCKET line of 218 bytes (100 body + 118 header).

- **flush=1:** ttfb / total / header: ttfb 0.21 s, total 1.72 s, Transfer-Encoding: chunked. The log shows Flush() requested and a 231-byte SOCKET line at the same timestamp (14:36:54.918992), then sleeping 1.5s, then handler returns, then a 5-byte SOCKET line (0\r\n\r\n).

- **Size of the SOCKET line(s):** 218 for flush=0, and 231 then 5 for flush=1.

Explain in your own words

1. **What does Flush() empty: A, B, or both? Use the log (the gap between the Flush line and the SOCKET line, and the size of the SOCKET line) as evidence.**

   Both. In the log, Flush() requested and the SOCKET conn.Write: 231 bytes line have the same timestamp, so the 100 bytes went from A through the chunkWriter and out of B into the socket in one step. If Flush() had only emptied A, the bytes would have stayed in B and no SOCKET line would have appeared. The 231 bytes are the 100 body bytes plus the header and chunk framing.

2. **Why did adding Flush() change the header from Content-Length to chunked? Which header moment decides it?**

   Moment 2, when the chunkWriter builds the header, decides it. With flush=0, A was emptied when the handler returned, so the length (100) was known and Go wrote Content-Length: 100. With flush=1, Flush() emptied A while the handler was still running (it still had a sleep ahead), so Go didn't know the total length and had to use chunked.

3. **A developer says: my handler returns quickly, so there is no need to flush. Under what condition is the developer right?**

   When the handler really has no slow work after it writes, so returning is almost immediate. In flush=0, the data only left when the handler returned. If that happens within a few milliseconds, the client wouldn't notice the wait, and you keep the Content-Length and the single write() call. The developer is wrong as soon as the handler does slow work (a database call, a sleep) before returning.

Experiment 5: The header you cannot take back

Part A: setting a header after the first Write

Prediction (does the client see X-Before? X-After?): The client sees X-Before but not X-After.

Actual: The client saw X-Before and did not see X-After. The server log shows now setting X-After, after the Write and no warning, because changing the map is legal but no longer has any effect.

Part B: the error that arrives too late (/late-error)

Prediction:

1. Status code, 200 or 500? Status 200.
2. Body: just the error text, or half page plus error text? The body is the half page plus the error text.
3. Will http.Error set Content-Type text/plain and nosniff? Not text/plain.
4. Will the log show any warning from Go? Yes, the log shows a Go warning.

Actual:

1. Status: 200
2. Body: the half page plus the error text (<html><body><h1>Report</h1>Internal Server Error)
3. http.Error did not set text/plain or nosniff. The Content-Type is text/html, guessed from the first bytes.

4. Yes. The log shows w.WriteHeader(500) TOO LATE: the status was already fixed, followed by Go's own http: superfluous response.WriteHeader call.

Part C: the safe pattern (/safe-error)

Prediction (status, Content-Type, body): Content type will be text/plain, status 500 and the body will contain the error.

Actual: Status 500 Internal Server Error, Content-Type: text/plain; charset=utf-8, X-Content-Type-Options: nosniff, body 'Internal Server Error'. The log shows w.WriteHeader(500) status fixed here with no TOO LATE and no superfluous warning.

Side-by-side comparison of /late-error and /safe-error

	/late-error	/safe-error
Status:	200 OK	500 Internal Server Error
Content-Type:	text/html	text/plain (set by http.Error)
nosniff:	missing	present
Body:	half page + error text	error text only
Server log warning:	yes (superfluous WriteHeader)	no

Part D: rewritten reportPage handler

```
func reportPage(w http.ResponseWriter, r *http.Request) {
    tmpl, err := template.ParseFiles("report.html")
    if err != nil {
        http.Error(w, "Internal Server Error", http.StatusInternalServerError)
        return
    }
    var buf bytes.Buffer
    if err := tmpl.Execute(&buf, data); err != nil {
        http.Error(w, "Internal Server Error", http.StatusInternalServerError)
        return
    }

    w.Header().Set("Content-Type", "text/html; charset=utf-8")
    buf.WriteTo(w)
}
```

Part E: explain in your own words

1. **In Part A the client had received nothing yet when X-After was set. Why was it already too late?**

   At moment 1, the first Write, Go took a copy of the header map. The later Set changed the original map, but the copy is what gets turned into header text. So it was too late even though the client had received nothing. Lateness is measured in header moments, not in bytes sent.

2. **In Part B, at which header moment did the status become 200, and which log line shows it?**

   Moment 1, at the handler's first Write. The log line is first Write: net/http calls WriteHeader(200) for you; status fixed here, just before w.Write #1: 27 bytes.

3. **Why does safeError get the right status when lateError does not? Answer in terms of before moment 1 and after moment 1.**

   In safeError, nothing was written to w before the error was found, because the page was built in a separate bytes.Buffer. So http.Error was the first call on w, and its WriteHeader(500) and header changes happened before moment 1, which is why they took effect. In lateError, the first Write had already fixed the status, so http.Error was after moment 1 and was ignored.

4. **Why is holding the whole response in memory a bad idea for a 500 MB file download? What could you check before the first Write instead?**

   It uses 500 MB of RAM for every request, so a few concurrent downloads could exhaust the server's memory. The client also waits until the entire file is loaded before getting any bytes. Instead, check what you can know in advance before the first Write, such as whether the file exists, whether the user is allowed to read it, and its size. Then stream it to w and send errors only before the first Write.

Experiment 6 (optional, Linux only)

Skipped (not running Linux)

Final exercises

Exercise A: /drip?size=700&count=5&pause=300

After this event	Bytes in A	Bytes in B	Sent to the kernel
Write #1 (700 bytes)	700	0	0
Write #2	1,400	0	0
Write #3 (crosses 2,048)	50+	Around 2,100+ (2,048 + Header)	0
Write #4	750+	2,190+	0
Write #5	1,450+	2,190+	0
The handler returns	0	0	3,640+

1. **How many SOCKET lines, and when (during the handler or after it returns)?**
   One SOCKET line, after handler returns. B peaks at about 3,650 bytes, which never reaches 4,096, so nothing leaves during the handler.

2. **Content-Length or chunked?**
   Chunked. A overflowed at Write #3 while the handler was still running, so the chunkWriter had to build the header without knowing the total length.

   Actual result and where your table disagreed with the log:

   One SOCKET line of 3,644 bytes (3,500 body + 144 header and chunk framing), after "handler returns", with Transfer-Encoding: chunked. My table matched the log: nothing reached the socket during the handler, because A overflowed once and B peaked below 4,096.

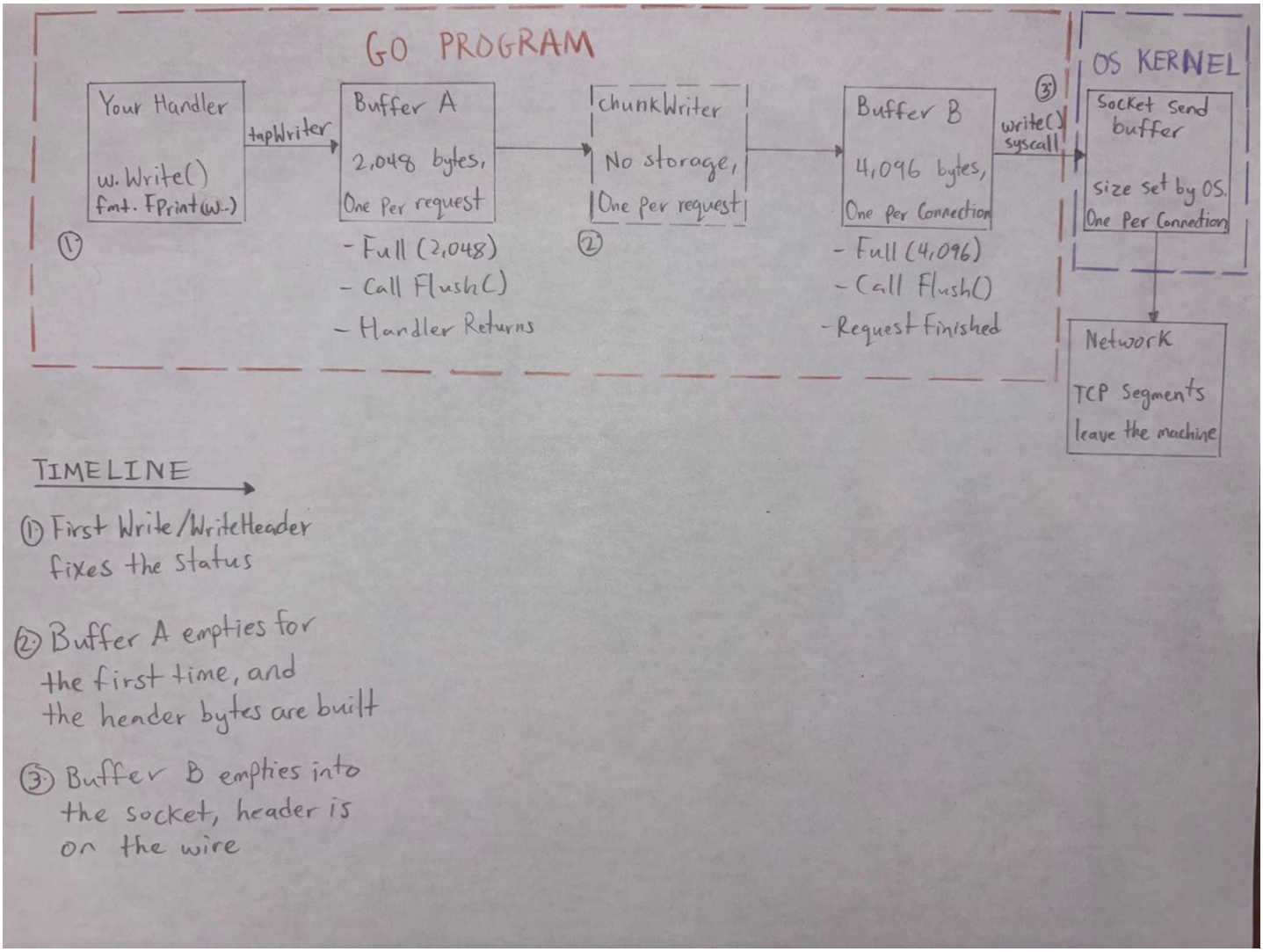
Exercise B: /drip?size=1500&count=6&pause=300

1. **At which Write number does the first SOCKET line appear?**
   Write #3, at about 0.6 s. It starts with HTTP/1.1 200 OK.
2. **Is there a second SOCKET line during the handler? At which Write?**
   Yes, a second one at Write #6, at about 1.5 s.
3. **How many SOCKET lines in total, and about how big is the last one?**
   Three SOCKET lines in total. They are 4,096, then 4,096, then a last one of about 970 bytes (the 808 left in A, plus the leftover in B and the end marker). The total is about 9,160, which is 9,000 plus about 160 for the header and chunk framing.

Actual result:

Three SOCKET lines: 4,096 bytes at Write #3 (starts HTTP/1.1 200 OK), 4,096 bytes at Write #6 (starts with cccc..., body data), and 973 bytes after "handler returns" (starts with ffff...). The total is 9,165 bytes. My predictions matched the log.

Exercise C: hand-drawn diagram



Reflection questions

1. **How many buffers hold your response bytes between w.Write() and the network? List them in order with sizes and owners.**

   There are three buffers between w.Write() and the network: Buffer A (2,048 bytes, one per request), Buffer B (4,096 bytes, one per connection, reused across requests) and the kernel send buffer (size set by the OS, one per connection). Between A and B sits the chunkWriter, which is not a buffer. It builds the header the first time A empties and then passes everything straight on.

2. **What three events empty Buffer A, and what three empty Buffer B? How do they differ?**

   A empties when it is full, when Flush() is called, or when the handler returns. B empties when it is full, when Flush() is called, or when the request is finished. They do different jobs: when A empties, its bytes pass through the chunkWriter into B, and the first time this happens the header is built (moment 2). When B empties, its bytes go to the kernel in one write() call (moment 3). Flush() empties both in one step. At the end of a request, the handler returning empties A, and then finishing the request empties B.

3. **Explain header moments 1, 2 and 3 using the log lines from Experiment 1 and Experiment 3 (/pieces?n=300).**

   Moment 1 is the status being fixed, shown by the log line first Write: net/http calls WriteHeader(200) for you; status fixed here. Moment 2 is the header being built when A first empties, which isn't in the log because it happens inside Go between the two instruments. Moment 3 is the header reaching the kernel, which is the first SOCKET conn.Write line.

   In /hello, moment 1 happens during the handler, then moments 2 and 3 happen right after it returns. The length was known, so the header says Content-Length: 26, and the 143-byte SOCKET line (26 body plus 117 header) shows the header was added on the way. In /pieces?n=300, the 3,000 bytes overflowed A during the handler, so moment 2 happened early and the header was chunked. B only held about 2,100 bytes, so the SOCKET line (3,144 bytes) didn't appear until after the handler returned. This shows moments 2 and 3 are separate events.

4. **A handler writes 100 bytes and then does a 5-second database query. What does the client see during those 5 seconds, what single line of code would change that, and what does it cost?**

   The client sees nothing for the whole 5 seconds, because the 100 bytes sit in Buffer A until the handler returns. /hold?flush=0 showed this, with a first byte after 1.72 s against a total of 1.72 s. Calling http.NewResponseController(w).Flush() right after the write sends the bytes at once, and in /hold?flush=1 the first byte arrived after 0.21 s while the total stayed the same. The cost is that the response loses its Content-Length and becomes chunked, because Flush() empties A while the handler is still running and the total length isn't known yet. The server also makes two system calls instead of one.

5. **Why can a handler not report a failure halfway through writing a response? Describe the safe pattern in your own words.**

   The first Write or WriteHeader fixes the status and copies the header map (moment 1), so a later http.Error is ignored. In /late-error the client got 200 OK with half a page plus the error text, a text/html Content-Type instead of text/plain, and no nosniff header. Only the server log showed the "superfluous WriteHeader" warning. The safe pattern is to build the response in a bytes.Buffer and not touch w until you know it will succeed. On failure, call http.Error as the first thing on w, which gave the correct 500 in /safe-error. On success, call buf.WriteTo(w).

6. **Why can a slow client keep your handler from returning?**

   A write() to the kernel can't finish until the kernel has room for the bytes. If the client doesn't read, the kernel send buffer fills up, so w.Write blocks and the handler can't return until the connection is read from or dies. The lab's Experiment 6 explanation describes a client that requests a 4 MB response and never reads. The handler stayed stuck for 8 seconds, until the connection closed. This is backpressure, and it is why servers set write timeouts.

7. **Which experiment changed your mental model the most, and what did you believe before it?**

   Experiment 1 changed my model the most. I knew the SOCKET line wouldn't say 26, but I had no idea where the extra bytes came from. The log showed 143 bytes, my 26 body bytes plus a 117-byte header, so the header is built inside Go between my handler and the socket. I also used to think that once my handler called w.Write(), the response was sent. The log showed the handler returning with no SOCKET line yet, so the bytes were still in Buffer A and the client had received nothing. That gave me the model I used for the rest of the lab: Write only copies bytes into a buffer, and the header is added and sent later.

8. **One question you still have about the response path:**

   The lab says Server-Sent Events works by writing an event, calling Flush() and waiting for the next one. Since Flush() makes the response chunked and empties both buffers each time, how does a handler keep one response open for minutes, and what happens to the buffers and the connection when the client disconnects in the middle?

Submission checklist

- ☑ Predictions for Experiments 1 to 5, written before running, next to the real results
- ☑ Filled table from Exercise A and answers to Exercise B
- ☑ Hand-drawn diagram from Exercise C
- ☑ Answers to the explain questions in each experiment
- ☑ Answers to the reflection questions
- ☑ Rewritten reportPage handler (Experiment 5, Part D)
- ☐ (Optional) strace lines from Experiment 6, Part A (skipped)